Elevate: AI-Powered Resume Analyzer and Optimizer

Shravankumar Janawade |Rohit Biradar

GitHub Repo - GitHub Link

Work Completed So Far

We have a fully working prototype. A user pastes their resume or uploads it as a PDF, enters the target job description, and the system runs two analyses before showing results in a web dashboard.

The first is keyword matching. We built a lookup list of skill terms and used pattern matching to pull relevant phrases from the job description, then checked which ones appear in the resume. The output is a match percentage and a list of missing keywords.

The second analysis works at the level of meaning. Using Sentence-BERT (all-MiniLM-L6-v2), we convert every resume bullet and the job description into numerical vectors and compute cosine similarity between them. This allows the tool to correctly identify that a bullet like "cut page load time by 40%" is relevant to a job asking for "performance optimization," even though they share no words in common.

Both scores combine into one weighted result (40% keyword, 60% semantic). The dashboard shows the overall score, a per-bullet similarity breakdown, and a keyword gap summary. The tool also identifies the weakest bullets and rewrites them using rule-based logic: replacing vague language with action verbs, flagging bullets that have no measurable outcomes, and fixing awkward phrasing. A Flan-T5 LLM rewrite option is also integrated and can be switched on when needed. The backend is Python and Flask; the frontend is React with Vite.

Immediate Next Steps

• **Section-level scoring**. Semantic analysis currently runs one bullet at a time. We will extend it so whole sections like work experience and education can be evaluated against the full job description, giving users a wider view of where their resume is strong or weak.

• **LLM rewrite testing**. The Flan-T5 pipeline is integrated but turned off by default. We will enable it, run it on a range of bullet types, and compare its output with the rule-based rewrites to learn when each approach performs better.

• **Evaluation setup**. We need a clear way to measure scoring accuracy. We will build a small, labeled set of resume and job description pairs with known relevance levels and run precision and recall calculations against our output

Challenges and Adjustments

The biggest technical problem was a version conflict between PyTorch, torchvision, and the Transformers library. Each expected a different building of torch, which caused the application to fail on import. Rebuilding the environment from scratch and only installing what the project uses resolved the issue.

We also found through early testing that keyword matching alone penalizes well-written resumes that use synonyms or rephrase things naturally. A candidate using perfectly good language can still score low. This is why the semantic score carries more weight in the result, and we will keep tuning the balance as we test on more examples. No changes to the original project scope are needed.

Plan for the Next Month

- **Week 1:** Implement section-level semantic scoring and integrate it into the dashboard. Begin building the labeled evaluation dataset with diverse resume and job description pairs.
- **Week 2:** Enable the Flan-T5 rewrite pipeline and run a structured comparison against rule-based suggestions across different resume styles and job categories to figure out how best to combine the two.
- **Week 3:** Compute precision and recall on the scoring system using the labeled dataset. Add a fairness check to catch cases where the keyword matcher penalizes certain phrasing styles without good reason.
- **Week 4:** Polish the frontend with cleaner visualizations and an iterative editing flow where users can apply a suggested rewrite and immediately re-score without starting over. Close out with final testing, bug fixes, and report writing.